Security Assessment

Final Report

ROYCO

Royco Dawn

April 2026

Prepared for Royco

certora Royco

Table of Contents

certora Royco

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Royco Dawn	https://github.com/roycoprotocol/royco-dawn/tree/feat/entry-point	85bf7dc (initial commit) c166255 (fix commit)	Solidity

Project Overview

This document describes the security review of **Royco Dawn**. The work was undertaken from **March 30th, 2026** to **April 16th, 2026**.

The following contract list is included in our scope:

```
src/auth/RoycoAuth.sol
src/base/RoycoBase.sol
src/libraries/Constants.sol
src/libraries/Types.sol
src/libraries/UtilsLib.sol
src/libraries/Units.sol
src/periphery/RoycoMarketSyncer.sol
src/periphery/RoycoEntryPoint.sol
src/accountant/RoycoAccountant.sol
src/factory/RoycoFactory.sol
src/factory/RolesConfiguration.sol
src/tranches/RoycoSeniorTranche.sol
src/tranches/RoycoJuniorTranche.sol
src/tranches/base/RoycoVaultTranche.sol
src/ydm/AdaptiveCurveYDM_V2.sol
src/kernels/base/RoycoKernel.sol
src/kernels/base/quoter/IdenticalERC4626SharesToAdminOracleQuoter.sol
```

```
src/kernels/base/quoter/IdenticalMakinaSharesToAdminOracleQuoter.sol
src/kernels/base/quoter/IdenticalERC4626SharesToChainlinkOracleQuoter.sol
src/kernels/base/quoter/IdenticalAssetsChainlinkToAdminOracleQuoter.sol
src/kernels/base/quoter/base/IdenticalAssetsOracleQuoter.sol
src/kernels/base/quoter/base/IdenticalAssetsChainlinkOracleQuoter.sol
src/kernels/base/quoter/base/IdenticalERC4626SharesOracleQuoter.sol
src/kernels/base/quoter/base/IdenticalAssetsAdminOracleQuoter.sol
src/kernels/Identical_Makina_ST_JT_MachineToAdminOracle_Kernel.sol
src/kernels/sUSDai_ST_JT_RedemptionSharePriceToAdminOracle_Kernel.sol
src/kernels/MaplePoolV2_ST_JT_ExitSharePriceToChainlinkOracle_Kernel.sol
src/kernels/Identical_ERC20_ST_JT_ChainlinkToAdminOracle_SoulBoundTrancheS
hares_Kernel.sol
src/kernels/ReUSD_ST_JT_ICLOracle_Kernel.sol
src/kernels/Identical_ERC4626_ST_JT_SharePriceToChainlinkOracle_Kernel.sol
src/kernels/Identical_AA_IdleCDO_ST_JT_VirtualPriceOracle_Kernel.sol
src/kernels/sUSDat_ST_JT_SharePriceToChainlinkOracle_Kernel.sol
src/kernels/Identical_ERC4626_ST_JT_SharePriceToAdminOracle_Kernel.sol
src/kernels/Identical_ERC20_ST_JT_ChainlinkToAdminOracle_Kernel.sol
```

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

Royco Dawn is a tranche-based structured product system where an ERC-20 vault, called Junior Tranche (JT), provides another vault (Senior Tranche, ST) with risk coverage against market fluctuation in exchange for a share of the yield accrued by the protected (ST) vault.

The protocol consists of a factory/access manager that is common to all markets, and multiple markets composed of senior and junior tranche ERC20 vaults, a kernel, an accountant, and YDM modules, with an async EntryPoint on top.

Once users deposit assets into the ST or JT, these are transferred to the kernel; the kernel then syncs the accountant, which marks raw net asset value (NAV) to market, tracks effective NAV and impermanent-loss style liabilities, enforces coverage, and allocates ST gains to JT through the yield distribution model (YDM).

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The main trust boundaries are the factory-backed role system, oracle/quoter admins, kernel/accountant admins, sync operators, and external integrations such as Chainlink and ERC4626-style vaults. The core invariants are NAV conservation, correct coverage/utilization enforcement, coherent fixed-term/perpetual state transitions, consistent delayed request valuation, and stable tranche share semantics under rounding.

The audit work focused most heavily on RoycoAccountant, RoycoKernel, RoycoVaultTranche, RoycoEntryPoint, the AdaptiveCurveYDM_V2 implementation, and the factory/roles setup, with cross-checks against deployment scripts and market configs. Special attention was given to the transition from permissioned participation by selected actors in the markets to a permissionless setup where anyone can participate.

Existing coverage in the repository is broad: accountant unit/comprehensive tests, YDM tests, abstract kernel fuzz suites, protocol-specific integration tests, entrypoint tests, and upgradeability/roles/pausability tests.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	1	1	-
Low	4	4	3
Informational	8	8	3
Total	13	13	6

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	JT deposits at jtEffectiveNAV = 0 can permanently brick JT recapitalizations	Medium	Acknowledged
L-01	Admin-controlled tranche share seizure is blocked when the tranche is paused	Low	Fixed
L-02	Self-liquidation bonus can increase utilization be incorrect for beta > 1	Low	Fixed
L-03	Missing TRANCHE_TYPE validation in RoycoFactory	Low	Fixed
L-04	Blacklisted users can exit escrowed tranche shares through `executeRedemption()`	Low	Acknowledged
I-01	RoycoEntryPoint could allow executions across multiple users	Informational	Fixed
I-02	Changes to RoycoEntryPoint's yieldRecipient are retroactive	Informational	Acknowledged
I-03	RoycoEntryPoint can force many dust-sized executions when close to tranche max	Informational	Acknowledged
I-04	Tranche deposits can succeed with zero shares	Informational	Fixed

I-05	Dust guard not re-checked after `jtNetGain` is reduced by coverage	Informational	Fixed
I-06	Unsafe cast in Units.toInt256	Informational	Acknowledged
I-07	Synchronous protocol fee collection allows DoS through fee recipient external blacklisting	Informational	Acknowledged
I-08	Soulbound tranche kernels are incompatible with RoycoEntryPoint	Informational	Acknowledged

Medium Severity Issues

M-01: JT deposits at jtEffectiveNAV = 0 can permanently brick JT recapitalizations

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: RoycoVaultTranche.sol	Status: Acknowledged	

Description:

When users deposit into vault tranches, the shares they get in return are calculated proportionally to the pre-existing shares (totalSupply) and net value of the tranche assets (effectiveNAVToMintAt).

For Junior tranches, this effectiveNAVToMintAt can become zero when the tranche has allocated all its value in support of its Senior tranche, while totalSupply remains unchanged from previous deposits.

This means that every time a recapitalization (a deposit with jtEffectiveNAV == 0 && totalSupply() > 0) happens, the JT share conversion rate gets diluted by 18+ orders of magnitude.

If over the lifetime of a market, a recapitalization event like this happens a few times (typically 3 or 4), the share price can become diluted to the point of overflowing even for reasonably-sized deposits.

Recommendations:

Consider resetting JT share ownerships under extreme dilution (the stRSR token is a known example of implementing the concept of “eras”).

Customer Response:

Acknowledged - There's no obstruction to existing LPs, obligations are still executed.

Low Severity Issues

L-01: Admin-controlled tranche share seizure is blocked when the tranche is paused

Severity: Low	Impact: Medium	Likelihood: Low
Files: RoycoVaultTranche.sol	Status: Fixed	

Description:

The seizeShares and seizeAndRedeemShares functions of the Royco tranches internally call ERC20PausableUpgradeable._update that is decorated with the whenNotPaused modifier.

This means that when a tranche is paused, these functions are not callable despite requiring a high privilege.

While it is not excluded that an admin can work around this limitation by multicalling unpause + seize + pause in a single transaction to perform the operation, it may not always be the case, i.e., if operating through an EOA.

Recommendations:

Consider having ERC20Upgradeable._update instead of super._update in seizeShares and seizeAndRedeemShares.

Customer Response:

Fixed in the 4772be2.

Fix Review:

Fix confirmed.

L-02: Self-liquidation bonus can increase utilization be incorrect for beta > 1

Severity: Low	Impact: Medium	Likelihood: Low
Files: RoycoKernel.sol	Status: Fixed	

Description:

The RoycoAccountant configuration parameters allow for betaWAD to be configured above 100%, and this conceptually makes sense in case JT and ST underlying are correlated through a leverage factor.

When betaWAD is configured above 100%, the math in RoycoKernel._computeMaxUtilizationNeutralBonus deviates from its theoretical formulation (RoycoKernel.sol:L920) because of the use of a defensive saturatingSub at L922, that is a non-linear component that is not present in the theoretical formula.

Because saturatingSub returns a higher value for WAD - beta than the theoretical math would, and its result is positively contributing to the returned max bonus, the resulting max bonus can be overestimated, which in turn can allow for the self-liquidation bonus to exceed the utilization-neutral threshold.

Recommendations:

Consider either linearizing the formula, i.e. with a a > b ? a - b : - (b - a) term, or capping beta to 100%, which is a safe choice for the current scope where all kernels are the same-asset.

Customer Response:

Fixed in the 034b9a9.

Fix Review:

Fix confirmed.

L-03: Missing TRANCHE_TYPE validation in RoycoFactory

Severity: Low	Impact: Medium	Likelihood: Low
Files: RoycoFactory.sol	Status: Fixed	

Description:

After deploying a market, RoycoFactory validates the creation and initialization of the various contracts against its expectations through its _validateDeployment function.

This function, however, does not validate that for the Junior and Senior tranches, a correct implementation (Junior or Senior, respectively) is being used.

Recommendations:

Consider adding to the _validateDeployment function checks for the values returned by _roycoMarket.seniorTranche.TRANCHE_TYPE() and _roycoMarket.juniorTranche.TRANCHE_TYPE().

Customer Response:

Fixed in the f538f17.

Fix Review:

Fix confirmed.

L-04: Blacklisted users can exit escrowed tranche shares through `executeRedemption()`

Severity: Low	Impact: Medium	Likelihood: Low
Files: RoycoEntryPoint.sol	Status: Acknowledged	

Description:

requestRedemption() on RoycoEntryPoint transfers the user's tranche shares into the contract for the duration of redemptionDelaySeconds. At execution time, executeRedemption() calls IRoycoVaultTranche(tranche).redeem(userSharesRedeemed, _receiver, address(this)) via _redeemWithYieldRouting(), so the kernel's preTrancheBalanceUpdateHook() is invoked with _caller = EntryPoint, _from = EntryPoint and _to = address(0). The original requester (_user) is never part of the call graph, and the underlying assets paid out by the kernel's _withdrawAssets() to _receiver are plain ERC20 transfers that do not invoke the hook at all.

As a result, if an admin blacklists the requester after they have queued a redemption, the existing request remains fully executable. The blacklisted user (or any keeper on their behalf, given the delay has elapsed) can still call executeRedemption() and withdraw the underlying ST/JT assets to _receiver, defeating the kernel's blacklist as a compliance control for any account with a pending redemption.

Recommendations:

In executeRedemption(), explicitly check that both _user (the original requester) and request.baseRequest.receiver are not blacklisted on the tranche's kernel before redeeming the escrowed shares.

Customer Response:

Acknowledged - Very unlikely a blacklist happens between request and execution. Also, we can work with the protocol to trace funds by tracking the actual receiver.

Informational Severity Issues

I-01: RoycoEntryPoint could allow executions across multiple users

Files:

RoycoEntryPoint.sol

Description:

The executeDeposits and executeRedemptions functions of RoycoEntryPoint accept multiple operations of a single user for batch execution.

If these functions accepted users in an array instead of a single address, batching could be further optimized to span several accounts without requiring multiple calls.

Recommendations:

Consider modifying the executeDeposits and executeRedemptions functions to accept users in an array.

Customer Response:

Fixed in the 034b9a9.

Fix Review:

Fix confirmed.

I-02: Changes to RoycoEntryPoint's yieldRecipient are retroactive

Files:

RoycoEntryPoint.sol

Description:

The RoycoEntryPoint contract allows admins to change the yield recipient (modifyTrancheConfigs) of a given tranche. This change, however, has a retroactive impact on requests that are posted but not yet executed at the time of the config modification.

Recommendations:

Consider snapshotting the yieldRecipient config with the user requests.

Customer Response:

Acknowledged. Expected behavior.

I-03: RoycoEntryPoint can force many dust-sized executions when close to tranche max

Files:

RoycoEntryPoint.sol

Description:

The deposit and redemption logic in RoycoEntryPoint caps executions at the tranche's declared maxDeposit and maxRedeem.

When this cap is applied, and in case a fee is defined for the operation, the amount effectively deposited or redeemed can be further reduced by the shares kept as fee.

This has two consequences - the most immediate one is that an unnecessarily smaller share of the operation is executed. In case of batched requests though, the portion of maxDeposit (or maxRedeem) that was unused because of fees will most likely be allocated to the next transaction, which will deposit that amount minus fees. Because fees are orders of magnitude smaller than requests, a batch close to the Tranch maximum will see its requests executed with a fat-tail of smaller and smaller amounts, quickly reaching the point where they become irrelevant.

Recommendations:

Consider adjusting the logic that caps to maxDeposit and maxRedeem to apply the cap to the amount after, not before, the fee, so the cap can be consumed by the earliest elements of the batch, in full when applicable.

Customer Response:

Acknowledge for now.

I-04: Tranche deposits can succeed with zero shares

Files:

RoycoVaultTranche.sol

Description:

The RoycoVaultTranche.deposit function implements a safeguard for preventing users from depositing ZERO_NAV_UNITS of value allocated.

Because they are not specifying a minSharesOut requirement, it would be desirable that shares are validated to be non-zero, because a non-zero allocated value can still translate to zero shares after the _convertToShares conversion is applied.

Recommendations:

Consider replacing the valueAllocated != ZERO_NAV_UNITS requirement with a slightly stricter shares != 0.

Customer Response:

Fixed in the 034b9a9.

Fix Review:

Fix confirmed.

I-05: Dust guard not re-checked after `jtNetGain` is reduced by coverage

Files:

RoycoAccountant.sol

Description:

RoycoAccountant._previewSyncTrancheAccounting() checks jtNetGain > jtNAVDustTolerance before computing JT protocol fees, but when later ST-loss coverage reduces jtNetGain via saturatingSub, the code recomputes jtProtocolFeeAccrued without reapplying the dust guard. As a result, a gain that was initially above dust can be pushed back into the dust range and still flow through fee math, violating the intended "no fees on dust" logic. In practice the impact is negligible because floor rounding usually collapses the fee to zero, but the accounting logic is internally inconsistent.

Recommendations:

Consider re-applying the dust guard when jtProtocolFeeAccrued is calculated.

Customer Response:

Fixed in the b090cc6.

Fix Review:

Fix confirmed.

I-06: Unsafe cast in Units.toInt256

Files:

Units.sol

Description:

The Units.toInt256 function performs a cast from NAV_UNIT (uint256) to int256. This cast can be lossy if the converted _unit exceeds type(int256).max.

While we don't see an extreme scenario where this loss can happen, we recommend defending in depth against overflows.

Recommendations:

Consider explicitly checking for _units exceeding type(int256).max.

Customer Response:

Acknowledged - Should never occur in a properly functioning market. Also, it will revert and we can upgrade contracts.

I-07: Synchronous protocol fee collection allows DoS through fee recipient external blacklisting

Files:

RoycoKernel.sol

Description:

In the _preOpSyncTrancheAccounting hook, called in all deposit and redeem operations, fees are collected and synchronously minted to the protocolFeeRecipient address.

This minting will, in turn, validate the operation through the kernel's _preTrancheBalanceUpdate that checks for protocolFeeRecipient to be an acceptable recipient for the underlying asset.

This means that if protocolFeeRecipient is at any point blacklisted or in any other way disallowed to receive amounts of underlying asset, the whole market is frozen as the fee minting fails, bubbling up the revert into _preOpSyncTrancheAccounting and the liquidity operation that triggered it.

Recommendations:

Consider minting fees synchronously to the Kernel instead, and allowing protocolFeeRecipient to pull them asynchronously at a later point. This would reduce the system-wide exposure to blacklisting to what is strictly necessary, which is the tranches themselves.

Customer Response:

Acknowledged - The fee recipient will be the Royco Foundation.

I-08: Soulbound tranche kernels are incompatible with RoycoEntryPoint

Files:

Identical_ERC20_ST_JT_ChainlinkToAdminOracle_SoulBoundTrancheShares_Kernel.sol

Description:

The SoulBound kernel uses the _preTrancheBalanceUpdate hook to prevent transfer of tranche shares other than fee attributions.

This limitation is, however, too strict as it does not account for the case of markets requiring an intermediate RoycoEntryPoint transfer for entries and exits.

Recommendations:

Consider adding an optional RoycoEntryPoint extra exception to the _preTrancheBalanceUpdate checks.

Customer Response:

Acknowledged - Seize share functionality was only added for securitize RWAs and we won't be tranching those anymore due to compliance burden:
1) we won't be deploying soul bound markets,
2) the entry point flat out won't work with these kernels so no assets can ever be at risk.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.